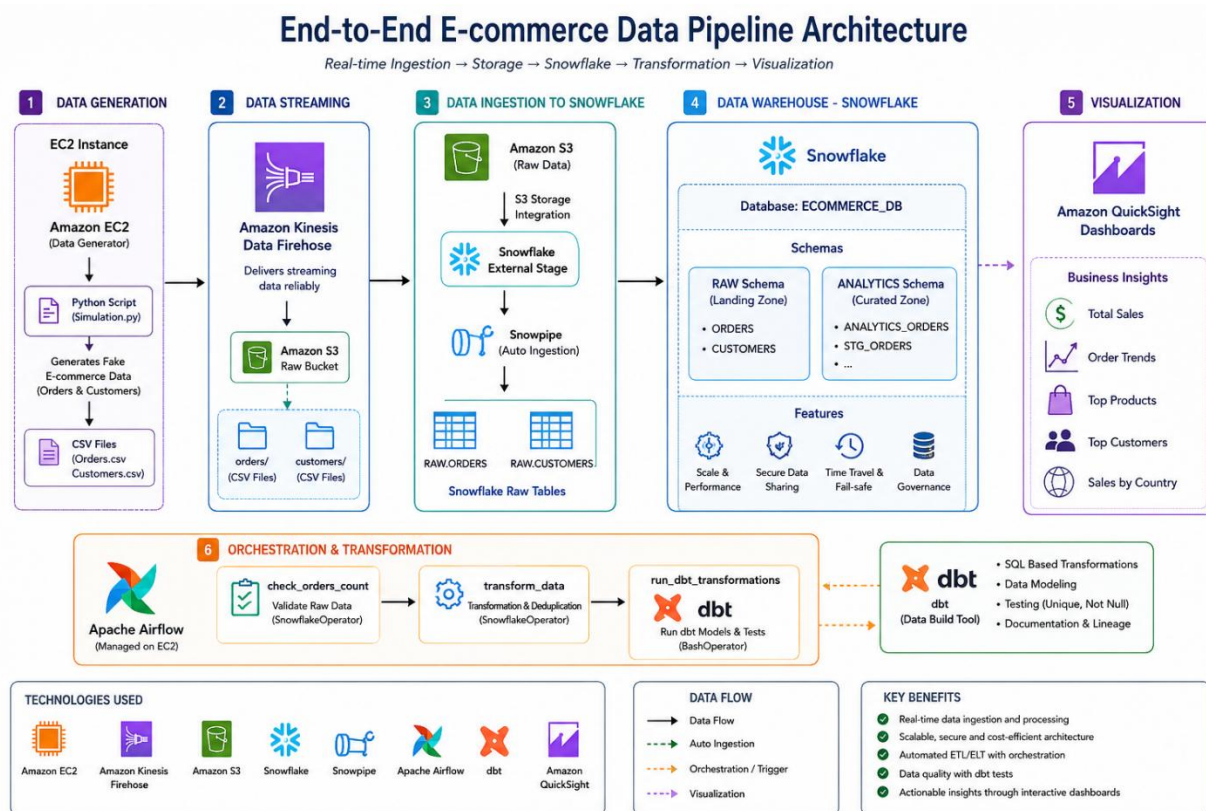


Ecommerce Data Pipeline using Snowflake and Managed Airflow on AWS



Ingestion (**Kinesis**)



Storage (**S3**)



Transformation (**Snowflake**)



Orchestration (**Airflow**)

PHASE 1: REAL-TIME DATA INGESTION PIPELINE

Architecture Overview

This phase establishes an automated, real-time streaming data ingestion layer. Synthetic e-commerce transactional data is generated on an EC2 instance, captured via the AWS Kinesis Agent, and delivered directly to Amazon S3 landing zones via Amazon Data Firehose.

Step 1: IAM Role Configuration

Create an IAM role to grant the EC2 instance the required permissions to stream logs and deliver data to target storage.

- **Role Name:** Kinesis-Agent-Role-Ecommerce
- **Trusted Entity:** AWS Service (EC2)
- **Attached Managed Policies:**
 - AmazonKinesisFirehoseFullAccess
 - CloudWatchFullAccess
 - AmazonS3FullAccess

Step 2: Add permissions Edit

Permissions policy summary

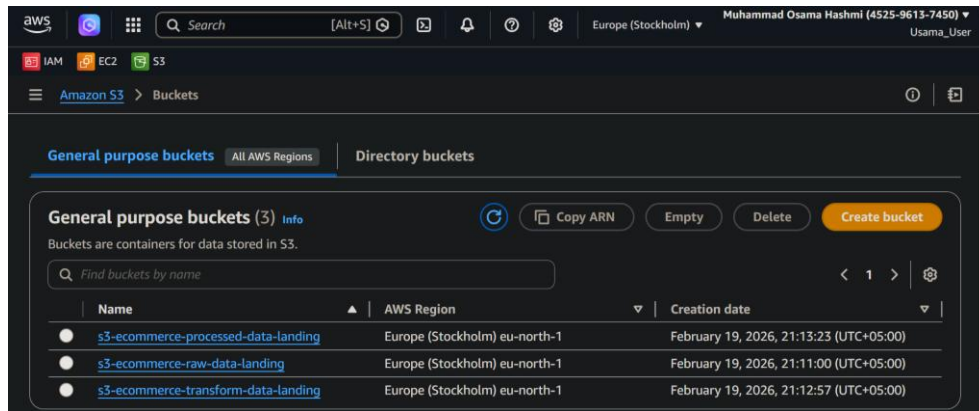
Policy name	Type	Attached as
AmazonKinesisFirehoseFullAccess	AWS managed	Permissions policy
AmazonS3FullAccess	AWS managed	Permissions policy
CloudWatchFullAccess	AWS managed	Permissions policy

Step 2: S3 Landing Zone & Storage Setup

Create three dedicated S3 storage layers to segregate raw, intermediate, and analytical datasets.

Buckets Creation

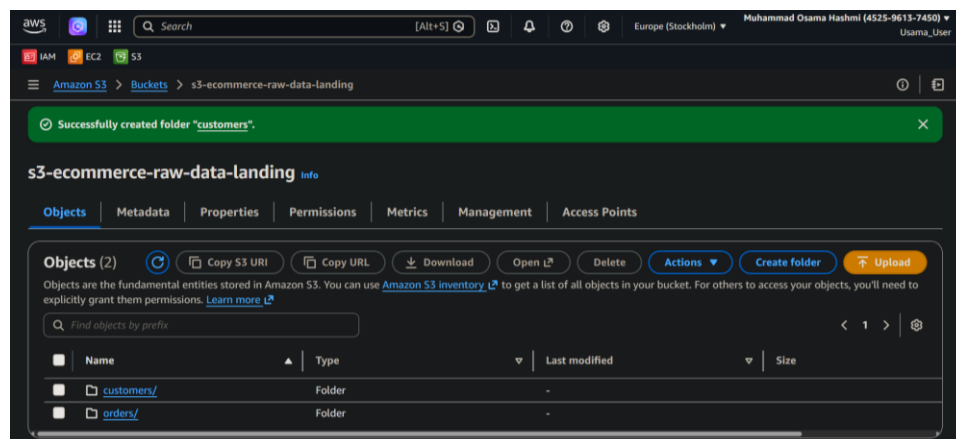
- **Raw Data Bucket:** s3-ecommerce-raw-data-landing
- **Transformed Data Bucket:** s3-ecommerce-transform-data-landing
- **Processed Data Bucket:** s3-ecommerce-processed-data-landing



Directory Structure (Raw Bucket)

Inside s3-ecommerce-raw-data-landing, create the following prefixes for data partition isolation:

- orders/
- customers/



Step 3: Amazon Data Firehose Stream Provisioning

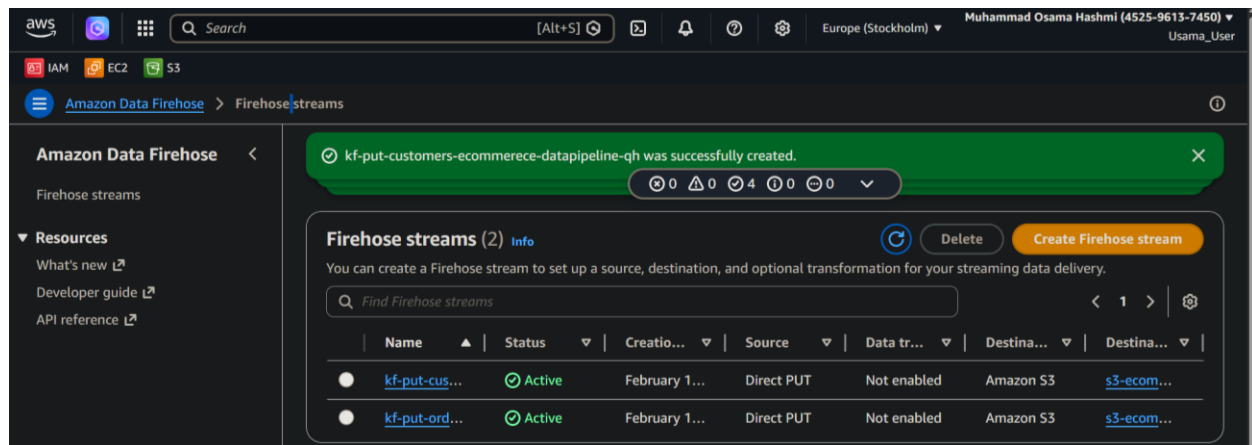
Configure delivery streams using **Direct PUT** as the source to handle real-time streaming ingested from the host agent.

Stream 1: Orders Data

- **Source:** Direct PUT
- **Destination:** Amazon S3
- **Firehose Stream Name:** kf-put-orders-ecommerce-datapipeline-oh
- **Target S3 Bucket:** s3-ecommerce-raw-data-landing
- **S3 Bucket Prefix:** orders/

Stream 2: Customers Data

- **Source:** Direct PUT
- **Destination:** Amazon S3
- **Firehose Stream Name:** kf-put-customers-ecommerce-datapipeline-oh
- **Target S3 Bucket:** s3-ecommerce-raw-data-landing
- **S3 Bucket Prefix:** customers/



Step 4: EC2 Data Generation Server Setup

Provision the host instance to run data generation workloads and host the streaming client agent.

1. **Instance Name:** Ecommerce-Data-Server
2. **AMI:** Amazon Linux 2023 AMI
3. **Instance Type:** t3.medium (or t3.micro depending on availability/tier)
4. **Key Pair:** RSA key pair (.pem format) stored locally for secure SSH access.
5. **Network Settings:** Configure Security Group inbound rules to allow SSH traffic (TCP Port 22) restricted to My IP.
6. **IAM Instance Profile:** Attach Kinesis-Agent-Role-Ecommerce.

Package Installation

```
# Update system packages
sudo yum update -y

# Install AWS Kinesis Agent
sudo yum install -y aws-kinesis-agent
```

Step 6: Kinesis Agent Configuration

Modify the client daemon configuration at `/etc/aws-kinesis/agent.json` to map local streaming files to target Firehose delivery streams.

```
sudo nano /etc/aws-kinesis/agent.json

{
  "cloudwatch.emitMetrics": true,
  "firehose.endpoint": "firehose.eu-north-1.amazonaws.com",
  "flows": [
    {
      "filePattern": "/tmp/orders.csv*",
      "deliveryStream": "kf-put-orders-ecommerce-datapipeline-oh",
      "initialPosition": "START_OF_FILE"
    },
    {
      "filePattern": "/tmp/customers.csv*",
      "deliveryStream": "kf-put-customers-ecommerce-datapipeline-oh",
      "initialPosition": "START_OF_FILE"
    }
  ]
}
```

Note: Update the `firehose.endpoint` value to match your designated AWS region.

Step 7: Service Initialization & Directory Privileges

```
# Set write/read permissions for log output destination
sudo chmod 777 /tmp

# Start and enable Kinesis Agent daemon
sudo service aws-kinesis-agent start
sudo chkconfig aws-kinesis-agent on
```

Step 8: Python Simulation Environment & Script Creation

Python Dependencies Setup

```
sudo yum install python3 python3-pip -y
pip3 install Faker
```

Data Generator Script (simulation.py)

Create a background generator script to emit continuous e-commerce record streams into local target files.

```
nano simulation.py
```

```
import csv
import time
import random
from faker import Faker
fake = Faker()

def generate_data():
    while True:
        # Generate and stream order record
        order_data = [
            random.randint(1000, 9999),
            fake.name(),
            random.choice(['Laptop', 'Phone', 'Watch']),
            random.randint(50, 500)
        ]
        with open('/tmp/orders.csv', 'a') as f:
            writer = csv.writer(f)
            writer.writerow(order_data)

        # Generate and stream customer record
        customer_data = [
            fake.name(),
            fake.email(),
            fake.country()
        ]
        with open('/tmp/customers.csv', 'a') as f:
            writer = csv.writer(f)
            writer.writerow(customer_data)

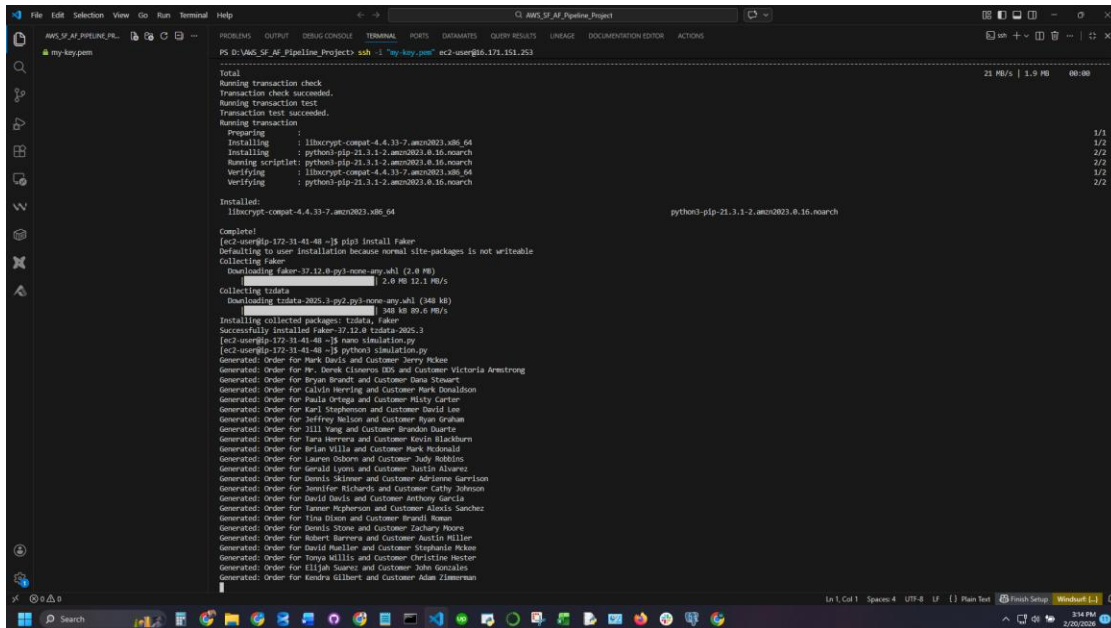
        print(f"Generated: Order for {order_data[1]} | Customer: {customer_data[0]}")
        time.sleep(2)

if __name__ == "__main__":
    generate_data()
```

Step 9: Pipeline Execution & Monitoring

Start Data Generation

python3 simulation.py

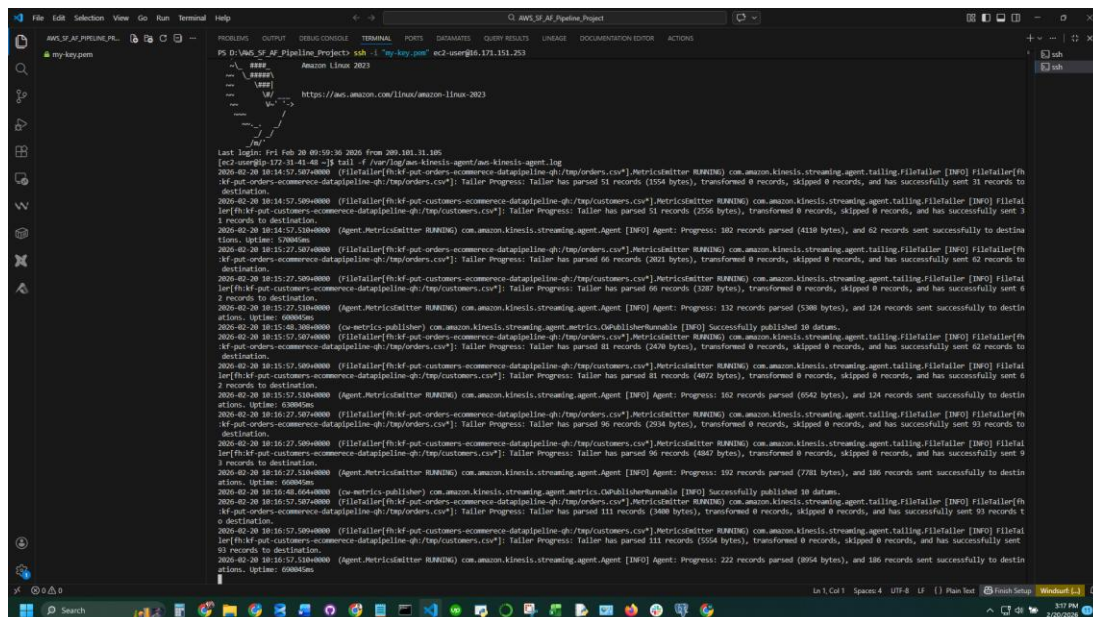


```
PS D:\AWS_SF_AF_Pipeline_Project> ssh -i "my-key.pem" ec2-user@65.171.151.251
Total
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Installing : libcrypt-compat-4.4.33-7.amzn2023.x86_64
  Installing : python3-pip-21.3.1-2.amzn2023.8.x86_64
  Running scriptlet: python3-pip-21.3.1-2.amzn2023.8.x86_64
  Verifying : libcrypt-compat-4.4.33-7.amzn2023.x86_64
  Verifying : python3-pip-21.3.1-2.amzn2023.8.x86_64
Installed:
  libcrypt-compat-4.4.33-7.amzn2023.x86_64                                python3-pip-21.3.1-2.amzn2023.8.x86_64
Complete!
[ec2-user@ip-172-31-41-48 ~]$ pip3 install Faker
Defaulting to user installation because normal site-packages is not writable
Collecting Faker
  Downloading Faker-17.12.0-py2.py3-none-any.whl (2.0 MB)
  Collecting tzdata-2023.3-py2.py3-none-any.whl (348 kB)
Installing collected packages: tzdata, Faker
Successfully installed Faker-17.12.0 tzdata-2023.3
[ec2-user@ip-172-31-41-48 ~]$ python3 simulation.py
[ec2-user@ip-172-31-41-48 ~]$ python3 simulation.py
Generated: Order for Mr. Derek Cisneros DBS and Customer Victoria Armstrong
Generated: Order for Bryan Brandt and Customer Dana Stewart
Generated: Order for Calvin Herring and Customer Mark Donaldson
Generated: Order for Paula Ortega and Customer Misty Carter
Generated: Order for Karl Stephenson and Customer David Lee
Generated: Order for Jeffrey Nelson and Customer Ryan Graham
Generated: Order for Bill Yang and Customer Brandon Duarte
Generated: Order for Tara Herrera and Customer Kevin Blackburn
Generated: Order for Brian Villa and Customer Mark McDonald
Generated: Order for Lauren Osborn and Customer Judy Robbins
Generated: Order for Donald Lyon and Customer Justin Alvarez
Generated: Order for Dennis Skinner and Customer Adrienne Garrison
Generated: Order for Jennifer Richard and Customer Cathy Johnson
Generated: Order for David Davis and Customer Anthony Garcia
Generated: Order for Tanner McPherson and Customer Alexis Sanchez
Generated: Order for Nina Blum and Customer Brandt Jones
Generated: Order for Dennis Stone and Customer Zachary Moore
Generated: Order for Robert Barrena and Customer Austin Miller
Generated: Order for David Butler and Customer Stephanie McKee
Generated: Order for Tonya Willis and Customer Christine Hester
Generated: Order for Elijah Suarez and Customer John Gonzalez
Generated: Order for Kendra Gilbert and Customer Adam Zimmerman
```

Verify Agent Activity & Log Output

In a secondary terminal window, tail the Kinesis Agent log output:

tail -f /var/log/aws-kinesis-agent/aws-kinesis-agent.log



```
PS D:\AWS_SF_AF_Pipeline_Project> ssh -i "my-key.pem" ec2-user@65.171.151.251
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

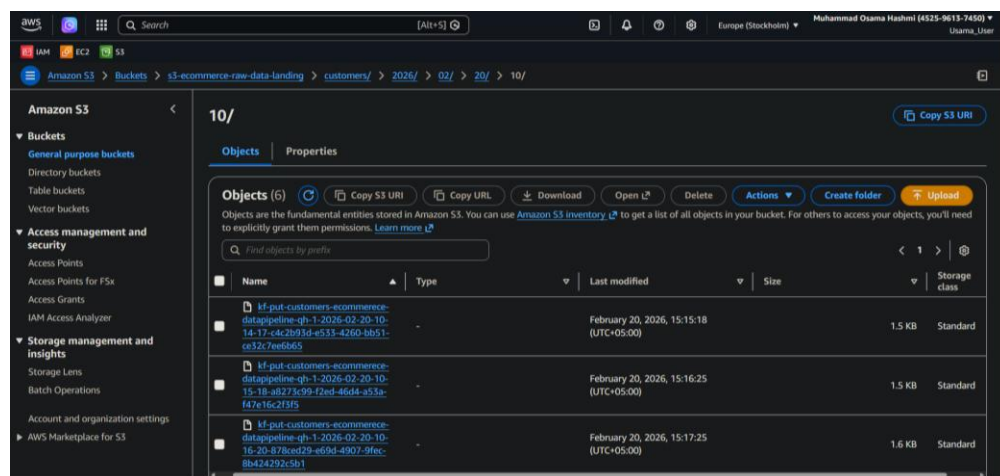
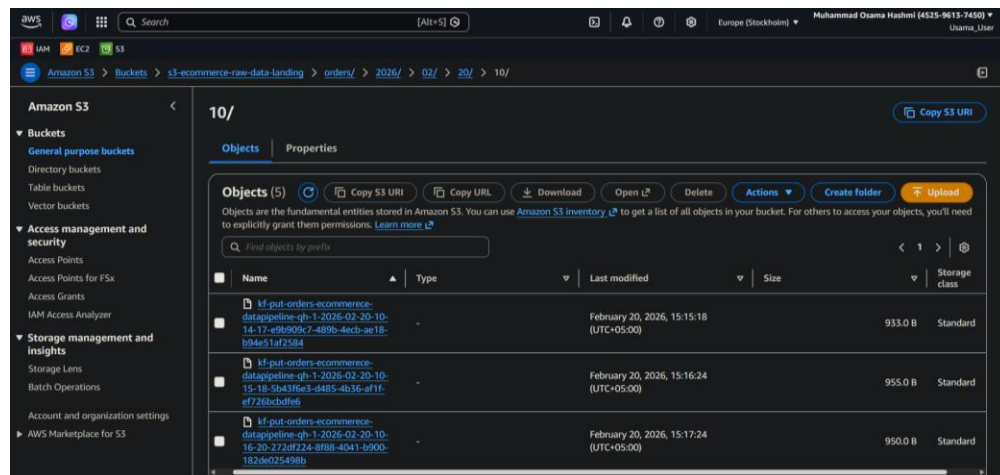
Last login: Fri Feb 20 09:59:56 2025 from 200.181.31.185
[ec2-user@ip-172-31-41-48 ~]$ tail -f /var/log/aws-kinesis-agent/aws-kinesis-agent.log
2025-02-20 10:14:57.507400000 [FileTailor(hk4f-put-orders-commerce-datapipeline-qi/tp/orders.cs*)MetricsEmitter RUNNING] com.amazonaws.kinesis.streaming.agent.tailing.FileTailor [INFO] FileTailor(hk4f-put-orders-commerce-datapipeline-qi/tp/orders.cs*) Tailor Progress: Tailor has parsed 51 records (3554 bytes), transformed 0 records, skipped 0 records, and has successfully sent 31 records to destination.
2025-02-20 10:14:57.509400000 [FileTailor(hk4f-put-orders-commerce-datapipeline-qi/tp/orders.cs*)MetricsEmitter RUNNING] com.amazonaws.kinesis.streaming.agent.tailing.FileTailor [INFO] FileTailor(hk4f-put-orders-commerce-datapipeline-qi/tp/orders.cs*) Tailor Progress: Tailor has parsed 51 records (3554 bytes), transformed 0 records, skipped 0 records, and has successfully sent 31 records to destination.
2025-02-20 10:14:57.510400000 [AgentMetricsEmitter RUNNING] com.amazonaws.kinesis.streaming.agent.Agent [INFO] Agent: Progress: 102 records parsed (4110 bytes), and 62 records sent successfully to destination. Uptime: 570000ms
2025-02-20 10:14:57.510400000 [FileTailor(hk4f-put-orders-commerce-datapipeline-qi/tp/orders.cs*)MetricsEmitter RUNNING] com.amazonaws.kinesis.streaming.agent.tailing.FileTailor [INFO] FileTailor(hk4f-put-orders-commerce-datapipeline-qi/tp/orders.cs*) Tailor Progress: Tailor has parsed 66 records (3287 bytes), transformed 0 records, skipped 0 records, and has successfully sent 62 records to destination.
2025-02-20 10:14:57.510400000 [FileTailor(hk4f-put-orders-commerce-datapipeline-qi/tp/orders.cs*)MetricsEmitter RUNNING] com.amazonaws.kinesis.streaming.agent.tailing.FileTailor [INFO] FileTailor(hk4f-put-orders-commerce-datapipeline-qi/tp/orders.cs*) Tailor Progress: Tailor has parsed 66 records (3287 bytes), transformed 0 records, skipped 0 records, and has successfully sent 62 records to destination.
2025-02-20 10:14:57.510400000 [AgentMetricsEmitter RUNNING] com.amazonaws.kinesis.streaming.agent.Agent [INFO] Agent: Progress: 132 records parsed (5380 bytes), and 124 records sent successfully to destination. Uptime: 600000ms
2025-02-20 10:14:57.510400000 [co-metrics-publisher] com.amazonaws.kinesis.streaming.agent.metrics.OpPublisherRunnable [INFO] Successfully published 10 datums.
2025-02-20 10:14:57.510400000 [FileTailor(hk4f-put-orders-commerce-datapipeline-qi/tp/orders.cs*)MetricsEmitter RUNNING] com.amazonaws.kinesis.streaming.agent.tailing.FileTailor [INFO] FileTailor(hk4f-put-orders-commerce-datapipeline-qi/tp/orders.cs*) Tailor Progress: Tailor has parsed 81 records (2430 bytes), transformed 0 records, skipped 0 records, and has successfully sent 81 records to destination.
2025-02-20 10:14:57.510400000 [FileTailor(hk4f-put-orders-commerce-datapipeline-qi/tp/orders.cs*)MetricsEmitter RUNNING] com.amazonaws.kinesis.streaming.agent.tailing.FileTailor [INFO] FileTailor(hk4f-put-orders-commerce-datapipeline-qi/tp/orders.cs*) Tailor Progress: Tailor has parsed 81 records (2430 bytes), transformed 0 records, skipped 0 records, and has successfully sent 81 records to destination.
2025-02-20 10:14:57.510400000 [AgentMetricsEmitter RUNNING] com.amazonaws.kinesis.streaming.agent.Agent [INFO] Agent: Progress: 162 records parsed (6642 bytes), and 124 records sent successfully to destination. Uptime: 630000ms
2025-02-20 10:14:57.510400000 [FileTailor(hk4f-put-orders-commerce-datapipeline-qi/tp/orders.cs*)MetricsEmitter RUNNING] com.amazonaws.kinesis.streaming.agent.tailing.FileTailor [INFO] FileTailor(hk4f-put-orders-commerce-datapipeline-qi/tp/orders.cs*) Tailor Progress: Tailor has parsed 96 records (2904 bytes), transformed 0 records, skipped 0 records, and has successfully sent 93 records to destination.
2025-02-20 10:14:57.510400000 [FileTailor(hk4f-put-orders-commerce-datapipeline-qi/tp/orders.cs*)MetricsEmitter RUNNING] com.amazonaws.kinesis.streaming.agent.tailing.FileTailor [INFO] FileTailor(hk4f-put-orders-commerce-datapipeline-qi/tp/orders.cs*) Tailor Progress: Tailor has parsed 96 records (2904 bytes), transformed 0 records, skipped 0 records, and has successfully sent 93 records to destination.
2025-02-20 10:14:57.510400000 [AgentMetricsEmitter RUNNING] com.amazonaws.kinesis.streaming.agent.Agent [INFO] Agent: Progress: 192 records parsed (7781 bytes), and 186 records sent successfully to destination. Uptime: 660000ms
2025-02-20 10:14:57.510400000 [co-metrics-publisher] com.amazonaws.kinesis.streaming.agent.metrics.OpPublisherRunnable [INFO] Successfully published 10 datums.
2025-02-20 10:14:57.510400000 [FileTailor(hk4f-put-orders-commerce-datapipeline-qi/tp/orders.cs*)MetricsEmitter RUNNING] com.amazonaws.kinesis.streaming.agent.tailing.FileTailor [INFO] FileTailor(hk4f-put-orders-commerce-datapipeline-qi/tp/orders.cs*) Tailor Progress: Tailor has parsed 111 records (3480 bytes), transformed 0 records, skipped 0 records, and has successfully sent 93 records to destination.
2025-02-20 10:14:57.510400000 [FileTailor(hk4f-put-orders-commerce-datapipeline-qi/tp/orders.cs*)MetricsEmitter RUNNING] com.amazonaws.kinesis.streaming.agent.tailing.FileTailor [INFO] FileTailor(hk4f-put-orders-commerce-datapipeline-qi/tp/orders.cs*) Tailor Progress: Tailor has parsed 111 records (3480 bytes), transformed 0 records, skipped 0 records, and has successfully sent 93 records to destination.
2025-02-20 10:14:57.510400000 [AgentMetricsEmitter RUNNING] com.amazonaws.kinesis.streaming.agent.Agent [INFO] Agent: Progress: 222 records parsed (8954 bytes), and 186 records sent successfully to destination. Uptime: 690000ms
```


Key Status Indicators

- **Tailing Status:** Confirms active file tracking (Tailing file: /tmp/orders.csv).
- **Ingestion Metrics:** Successful payload deliveries emitted in log entries (Successfully sent X records to destination).

Step 10: S3 Target Validation

1. Open the Amazon S3 Console and navigate to s3-ecommerce-raw-data-landing.
2. Access orders/ and customers/ prefixes.
3. Confirm automated partitioned subfolder creation structured by /YYYY/MM/DD/HH/.
4. Inspect delivered object files (prefixed with kf-put-orders-...) to ensure stream formatting matches expected schema payloads.



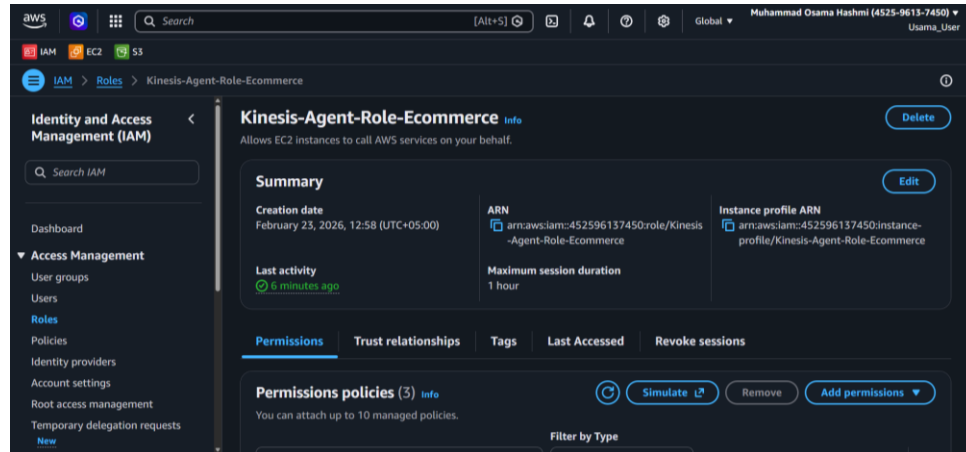
 **Congratulations! Your Phase 1 (Data Ingestion) is officially complete!**

PHASE 2: SNOWFLAKE SETUP & DATA LOADING

Now we move to the next phase, where we will load this S3 data into Snowflake.

Copy ARN

- Log in to the **AWS Console** and navigate to the **IAM (Roles)** service.
- Find the role you created: **Kinesis-Agent-Role-Ecommerce** and click on it.
- Once the role opens, you will see the **ARN** in Summary section at the very top. Copy it.



ARN: arn:aws:iam::.....:role/Kinesis-Agent-Role-Ecommerce

STEP 1: STORAGE INTEGRATION (SNOWFLAKE-S3 HANDSHAKE)

Access to your S3 Bucket

CREATE OR REPLACE STORAGE INTEGRATION s3_integration

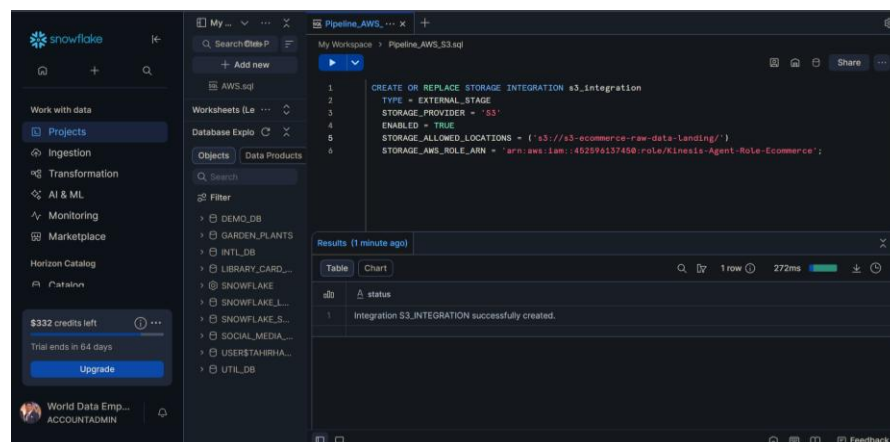
TYPE = EXTERNAL_STAGE

STORAGE_PROVIDER = 'S3'

ENABLED = TRUE

STORAGE_ALLOWED_LOCATIONS = ('s3://s3-ecommerce-raw-data-landing/')

STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::.....:role/Kinesis-Agent-Role-Ecommerce';



STEP 2: CONNECTION DETAILS HASIL KAREIN

DESC INTEGRATION s3_integration;

property	property_type	property_value	property_default
ENABLED	Boolean	true	false
STORAGE_PROVIDER	String	S3	
STORAGE_ALLOWED_LOCATIONS	List	s3://s3-e-commerce-raw-data-landing/	[]
STORAGE_BLOCKED_LOCATIONS	List		[]
STORAGE_AWS_IAM_USER_ARN	String	arn:aws:iam::194317476460:user/57cd1000-s	
STORAGE_AWS_ROLE_ARN	String	arn:aws:iam::452596137450:role/Kinesis-Agent-Role-Ecommerce	
STORAGE_AWS_EXTERNAL_ID	String	YOB72705_SFCRole=2_PsDN8knK+A92zoUQp4ZR9GF0JQ4=	
USE_PRIVATELINK_ENDPOINT	Boolean	false	false
COMMENT	String		

Note these 2 important things

1. **STORAGE_AWS_IAM_USER_ARN** (Snowflake ARN).
arn:aws:iam::194317476460:user/57cd1000-s
2. **STORAGE_AWS_EXTERNAL_ID** (A unique ID).
YOB72705_SFCRole=2_8LsYDJOSoDmMib44sH4CifWaleo=

STEP 3: UPDATE TRUST RELATIONSHIP IN AWS IAM ROLE

Now, go back to the **AWS Console** and follow these steps:

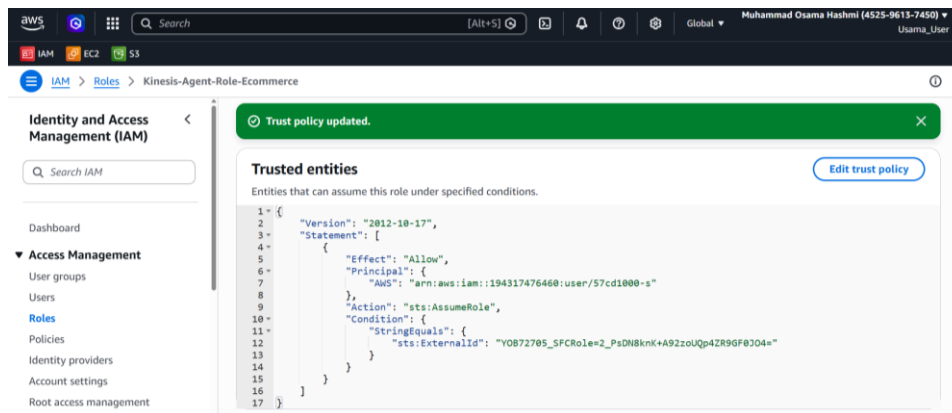
1. Navigate to **IAM (Roles)** and open the role you created (**Kinesis-Agent-Role-Ecommerce**).
2. Click on the **Trust Relationships** tab and then click on **Edit trust policy**.
3. Delete the old policy text and paste the new policy provided below.
4. Click on **Update policy**.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::194317476460:user/57cd1000-s"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

```

"Action": "sts:AssumeRole",
"Condition": {
  "StringEquals": {
    "sts:ExternalId": "YOB72705_SFCRole=4_aG8wCWUYcKIZFkG2/+bQB06iZ5Q="
  }
}
}
]
}

```



Snowflake will only be able to fetch your data after you run DESC INTEGRATION in Snowflake, extract those two (2) values, and update them in AWS.

The "handshake" between Snowflake and S3 is now complete! 🐼

Now, we will verify whether Snowflake can actually access and see your bucket.

1. Set the Database and Schema Context

First, tell Snowflake which database and schema you want to use.

```

-- Database aur Schema select karein
CREATE DATABASE ECOMMERCE_DB;
USE ECOMMERCE_DB;

CREATE SCHEMA RAW;
USE SCHEMA RAW;

```

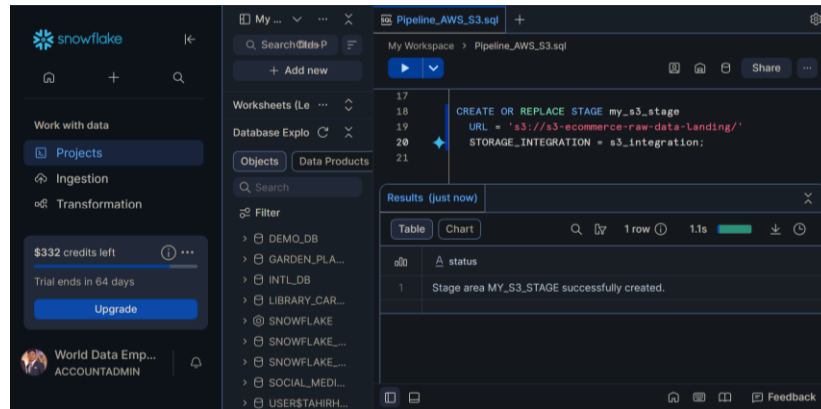
2. Create External Stage

The integration object has been created. Now, we need to create a **Stage** that will utilize this integration. This will make loading the data much easier:

```

-- S3 ke liye Stage banana
CREATE OR REPLACE STAGE my_s3_stage
  URL = 's3://s3-ecommerce-raw-data-landing/'
  STORAGE_INTEGRATION = s3_integration;

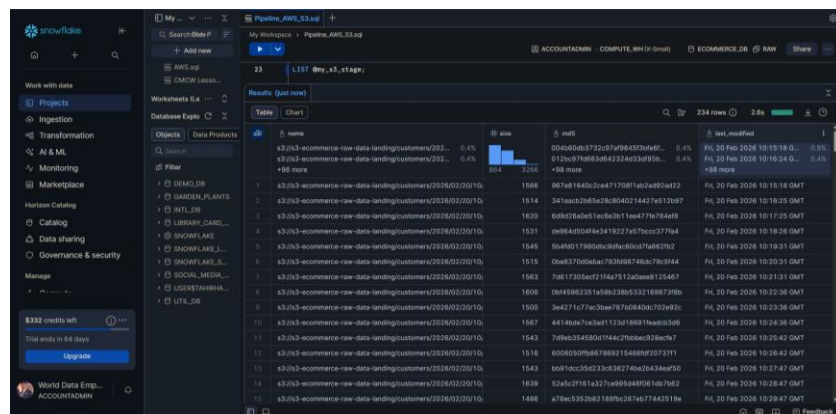
```



3. Connection Test

Now, run the command below to list and view your files in S3:

```
LIST @my_s3_stage;
```



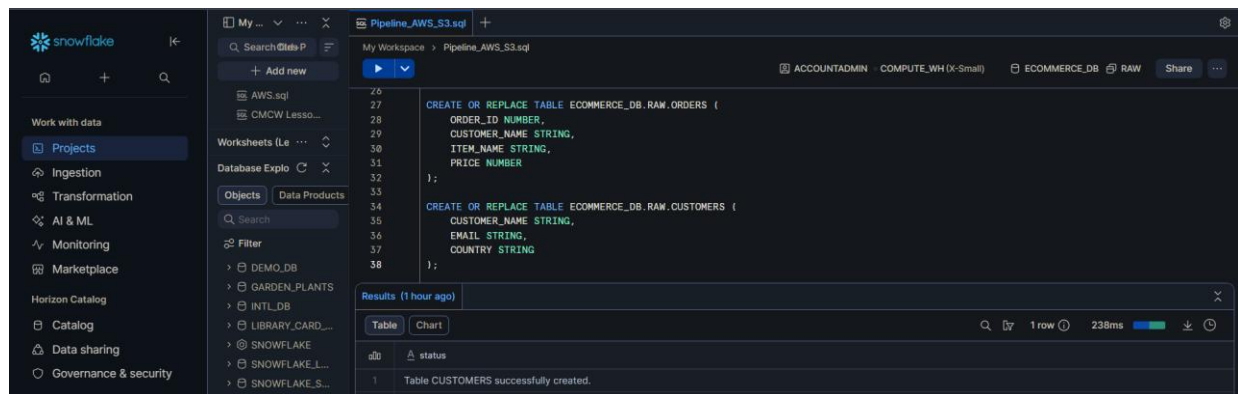
4. Create Tables for Orders & Customers

For Orders Table: Data format: ID, Name, Item, Price

```
CREATE OR REPLACE TABLE ECOMMERCE_DB.RAW.ORDERS (
  ORDER_ID NUMBER,
  CUSTOMER_NAME STRING,
  ITEM_NAME STRING,
  PRICE NUMBER
);
```

For Customers Table: Data format: Name, Email, Country

```
CREATE OR REPLACE TABLE ECOMMERCE_DB.RAW.CUSTOMERS (
  CUSTOMER_NAME STRING,
  EMAIL STRING,
  COUNTRY STRING
);
```



5. Snowpipe

In industry projects where data is streamed in real-time (just like your Kinesis Firehose is doing), **Snowpipe** is the absolute best solution.

Advantages of Using Snowpipe

- **Automation:** As soon as Firehose drops a new file into S3, Snowpipe instantly detects it and loads it into the target table.
- **Serverless:** You don't need to trigger or run commands manually; Snowflake completely manages the infrastructure and execution on the backend.
- **Efficiency:** It is highly optimized and cost-effective for continuously loading frequent, small batches of files.

Snowpipe Setup Karne ka Tarika:

➤ Step 1: Create Pipe

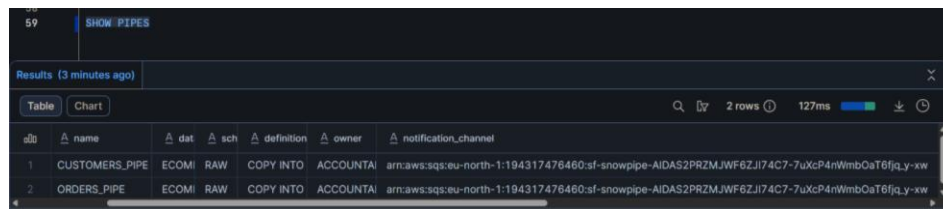
```
-- For Orders Pipe
CREATE OR REPLACE PIPE ECOMMERCE_DB.RAW.ORDERS_PIPE
AUTO_INGEST = TRUE
AS
COPY INTO ECOMMERCE_DB.RAW.ORDERS
FROM @my_s3_stage/orders/
FILE_FORMAT = (TYPE = CSV FIELD_DELIMITER = ',' SKIP_HEADER = 0);

-- For Customers Pipe
CREATE OR REPLACE PIPE ECOMMERCE_DB.RAW.CUSTOMERS_PIPE
AUTO_INGEST = TRUE
AS
COPY INTO ECOMMERCE_DB.RAW.CUSTOMERS
FROM @my_s3_stage/customers/
FILE_FORMAT = (TYPE = CSV FIELD_DELIMITER = ',' SKIP_HEADER = 0);
```

➤ Step 2: S3 Event Notification (The Trigger)

To notify Snowpipe that a new file has arrived, we need to configure an event notification on the S3 bucket:

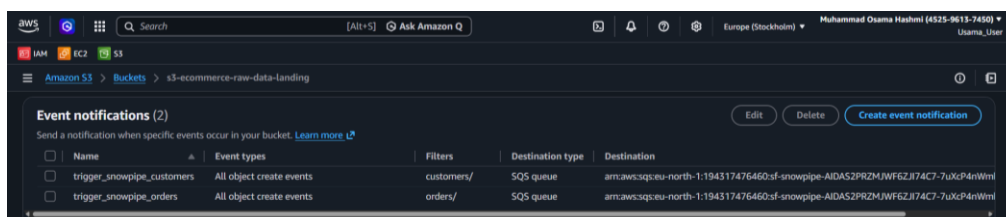
1. Run in Snowflake `SHOW PIPES;`
2. Copy ID of `notification_channel` column (`SQS QUEUE ARN`)



The screenshot shows the results of a 'SHOW PIPES' query in Snowflake. The table has 7 columns: name, dat, sch, definition, owner, and notification_channel. There are two rows of data.

	name	dat	sch	definition	owner	notification_channel
1	CUSTOMERS_PIPE	ECOM	RAW	COPY INTO	ACCOUNTA	arn:aws:sqs:eu-north-1:194317476460:sf-snowpipe-AIDAS2PRZMJWF6ZJI74C7-7uXcP4nWmbOaT6Jq.y-xw
2	ORDERS_PIPE	ECOM	RAW	COPY INTO	ACCOUNTA	arn:aws:sqs:eu-north-1:194317476460:sf-snowpipe-AIDAS2PRZMJWF6ZJI74C7-7uXcP4nWmbOaT6Jq.y-xw

3. AWS S3 Console raw bucket -> Properties tab -> Event notifications.
4. "Create event notification":
 - Event name: trigger_snowpipe_orders
 - Prefix: orders/
 - Event types: "All object create events"
 - Destination: "SQS Queue" select karein.
 - SQS Queue ARN: Past that ID.
 - Save Changes
 - Same with customers
 - Event name: trigger_snowpipe_customers
 - Prefix: customers/



The screenshot shows the AWS S3 Console 'Event notifications' page for a bucket named 's3-ecommerce-raw-data-landing'. It displays a table with 5 columns: Name, Event types, Filters, Destination type, and Destination. There are two event notifications listed.

Name	Event types	Filters	Destination type	Destination
trigger_snowpipe_customers	All object create events	customers/	SQS queue	arn:aws:sqs:eu-north-1:194317476460:sf-snowpipe-AIDAS2PRZMJWF6ZJI74C7-7uXcP4nWmbOaT6Jq.y-xw
trigger_snowpipe_orders	All object create events	orders/	SQS queue	arn:aws:sqs:eu-north-1:194317476460:sf-snowpipe-AIDAS2PRZMJWF6ZJI74C7-7uXcP4nWmbOaT6Jq.y-xw

Check Point:

Once you set this up, monitor your EC2 terminal to ensure data is actively being generated. Then, wait 1–2 minutes and run the following query in Snowflake to verify:

```
SELECT COUNT(*) FROM ORDERS;
```

You will notice that the row count automatically increases every time you re-run the command!

The screenshot shows a database management tool with a sidebar on the left containing categories like 'Governance & security', 'Manage', 'Compute', 'Postgres', and 'Admin'. The main panel displays a query 'SELECT COUNT(*) FROM ORDERS;' with a result table containing one row with the value 898. The interface is dark-themed.

Results (just now)
898

After 1 min

The screenshot shows the same database management tool as before, but the result of the query 'SELECT COUNT(*) FROM ORDERS;' is now 990. The interface remains dark-themed.

Results (just now)
990

PHASE 3: AIRFLOW SETUP

STEP 1: ON AIRFLOW DASHBOARD

Airflow is installed on your EC2 instance. Now you just need to run it and access it via your web browser.

Go to your EC2 terminal and run the following commands:

1. Reconnect to your EC2 Instance:

```
ssh -i "<your-key>.pem" ec2-user@<your-public-ip>
```

2. Create and Activate the Airflow Environment

```
python3 -m venv airflow_env
source airflow_env/bin/activate
```

3. Upgrade pip and Install Airflow with Snowflake Provider

```
pip install --upgrade pip

pip install "apache-airflow==2.8.1" apache-airflow-providers-snowflake --constraint
"https://raw.githubusercontent.com/apache/airflow/constraints-2.8.1/constraints-3.9.txt"
```

4. Setup Airflow Home and Initialize the Database

Home directory define

```
export AIRFLOW_HOME=~/.airflow
```

Database initialize

```
airflow db init
```

5. Create an Admin User (For Dashboard Login)

```
airflow users create \
  --username admin \
  --firstname Osama \
```

```
--lastname Hashmi \  
--role Admin \  
--email osamahashmi28@gmail.com \  
--password 'Admin#123'
```

6. Start the Webserver and Scheduler

Open two new terminal tabs on your local machine. In each tab, run the following commands first to connect to EC2 and set up your environment:

```
ssh -i "<your-key>.pem" ec2-user@<your-public-ip>  
  
source airflow_env/bin/activate  
  
export AIRFLOW_HOME=~/.airflow
```

- **Terminal 1:** `airflow scheduler -D`
- **Terminal 2:** `airflow webserver -D`

PHASE 3: WORKFLOW ORCHESTRATION & SNOWFLAKE AUTOMATION WITH APACHE AIRFLOW

Overview

Phase 3 establishes workflow orchestration by deploying Apache Airflow on the EC2 instance. This phase covers setting up a virtual environment, initializing the Airflow metadata database, creating admin credentials, configuring custom security rules for UI access, establishing a native Snowflake connection, and deploying a Directed Acyclic Graph (DAG) for automated task execution.

Phase 3 Architecture Workflow

[Airflow Webserver/Scheduler] — (Snowflake Hook/Operator) → [Snowflake Data Warehouse]



[ECOMMERCE_DB.RAW.ORDERS]

Step 1: Apache Airflow Environment & Service Provisioning

Go to your EC2 terminal and run the following commands:

1. Reconnect to your EC2 Instance:

```
ssh -i "<your-key>.pem" ec2-user@<your-public-ip>
```

2. Create and Activate the Airflow Environment

```
python3 -m venv airflow_env  
source airflow_env/bin/activate
```

3. Upgrade pip and Install Airflow with Snowflake Provider

```
pip install --upgrade pip  
  
pip install "apache-airflow==2.8.1" apache-airflow-providers-snowflake --constraint  
"https://raw.githubusercontent.com/apache/airflow/constraints-2.8.1/constraints-3.9.txt"
```

4. Setup Airflow Home and Initialize the Database

Home directory define

```
export AIRFLOW_HOME=~/.airflow
```

Database initialize

```
airflow db init
```

5. Create an Admin User (For Dashboard Login)

```
airflow users create \  
--username admin \  
--firstname Osama \  
--lastname Hashmi \  
--role Admin \  
--email osamahashmi28@gmail.com \  
--password 'Admin#123'
```

6. Start the Webserver and Scheduler

Open two new terminal tabs on your local machine. In each tab, run the following commands first to connect to EC2 and set up your environment:

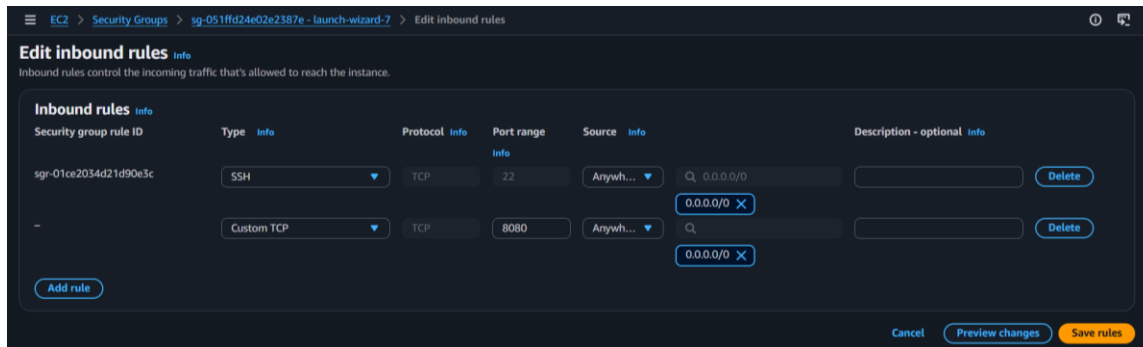
```
ssh -i "<your-key>.pem" ec2-user@<your-public-ip>  
  
source airflow_env/bin/activate  
  
export AIRFLOW_HOME=~/.airflow
```

- **Terminal 1:** `airflow scheduler -D`
- **Terminal 2:** `airflow webserver -D`

Step 2: Network Inbound Rule Configuration (AWS Security Group)

To allow web browser access to the Airflow Web UI on port 8080:

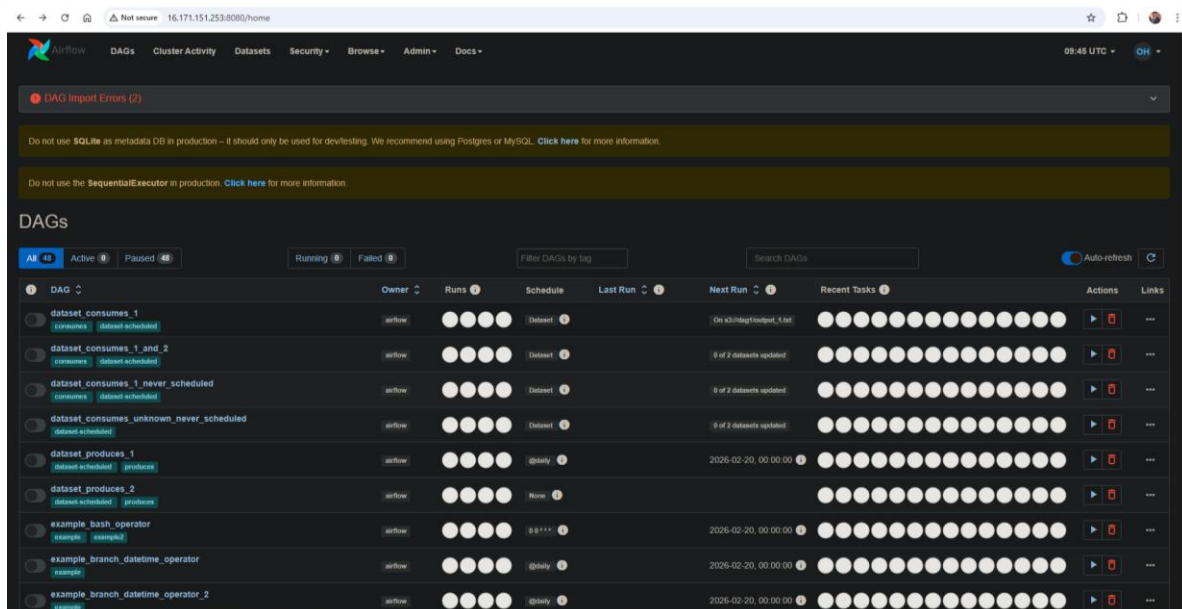
1. Open the **AWS Management Console** → **EC2 Instances**.
2. Select Ecommerce-Data-Server → Navigate to the **Security** tab → Click the active **Security Group**.
3. Under **Inbound Rules**, click **Edit inbound rules** and add:
 - **Type:** Custom TCP
 - **Port Range:** 8080
 - **Source:** 0.0.0.0/0 (Anywhere-IPv4)
4. Save the security group rule.



Access the UI in your browser:

<http://<your-public-ip>:8080>

Log in using the created administrator credentials (admin / Admin#123).



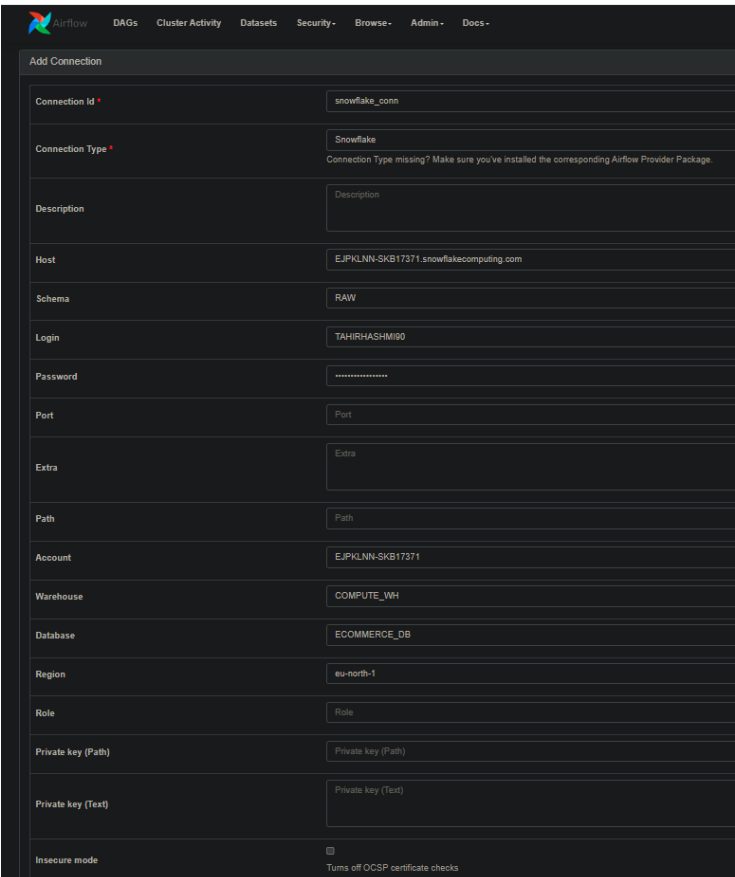
Step 3: Configure Snowflake Connection in Airflow UI

Establish cross-platform communication between *Apache Airflow* and *Snowflake* via the *Web UI*:

1. In the top navigation bar, select **Admin** → **Connections**.
2. Click the + (Add a new record) button.
3. Configure the parameters as follows:

Field	Value
Connection Id	snowflake_conn
Connection Type	Snowflake
Schema	RAW
Login	<your_snowflake_username>
Password	<your_snowflake_password>
Account	<your_snowflake_account_identifier>
Warehouse	COMPUTE_WH
Database	ECOMMERCE_DB
Region	us-west-2 (or your respective region)

4. Click **Save**.



The screenshot shows the 'Add Connection' form in the Airflow Web UI. The browser address bar indicates the URL is 56.228.12.216:8080/connection/add. The form is titled 'Add Connection' and contains the following fields and values:

- Connection Id: snowflake_conn
- Connection Type: Snowflake (with a warning message: 'Connection Type missing? Make sure you've installed the corresponding Airflow Provider Package.')
- Description: (empty)
- Host: EJPKLNN-SKB17371.snowflakecomputing.com
- Schema: RAW
- Login: TAHIRHASHMI90
- Password: (masked with dots)
- Port: Port
- Extra: (empty)
- Path: Path
- Account: EJPKLNN-SKB17371
- Warehouse: COMPUTE_WH
- Database: ECOMMERCE_DB
- Region: eu-north-1
- Role: Role
- Private key (Path): Private key (Path)
- Private key (Text): Private key (Text)
- Insecure mode: ☐ Turns off OCSP certificate checks

ID	Name	Provider	Conn ID	Schema	Login	Password	Extra
pinot_broker_default	pinot	localhost	9000	False	False		
postgres_default	postgres	postgres		False	False		
presto_default	presto	localhost	3400	False	False		
redis_default	redis	redis	6379	False	False		
redshift_default	redshift			False	False		
salesforce_default	salesforce			False	False		
segment_default	segment			False	False		
stp_default	stp	localhost	22	False	False		
snowflake_conn	snowflake	EJPKLNN-SKB17371.snowflakecomputing.com		False	False		
spark_default	spark	yarn		False	False		
sqlite_default	sqlite	/tmp/sqlite_default.db		False	False		
sqoop_default	sqoop	rdbms		False	False		
ssh_default	ssh	localhost		False	False		
tableau_default	tableau	https://tableau.server.url		False	False		
tabular_default	tabular	https://api.tabulardata.io/ws/v1		False	False		
trino_default	trino	localhost	3400	False	False		
vertica_default	vertica	localhost	5433	False	False		

Step 4: DAG Directory Initialization & Script Deployment

1. Create DAGs Directory

Return to the EC2 terminal session and create the DAG repository folder:

```
mkdir -p ~/airflow/dags
```

```
(airflow_env) [ec2-user@ip-172-31-28-66 ~]$ mkdir -p ~/airflow/dags
(airflow_env) [ec2-user@ip-172-31-28-66 ~]$ ls ~/airflow
airflow-webserver.pid  airflow.cfg  airflow.db  dags  logs  webserver_config.py
(airflow_env) [ec2-user@ip-172-31-28-66 ~]$
```

2. Create Automation Script (snowflake_automation.py)

```
nano ~/airflow/dags/snowflake_automation.py
```

Insert the following DAG definition code:

```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.providers.snowflake.operators.snowflake import SnowflakeOperator

# Default task configuration arguments
default_args = {
    'owner': 'Osama',
    'depends_on_past': False,
    'start_date': datetime(2024, 1, 1),
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
```

```
# DAG Definition
with DAG(
    'snowflake_test_dag',
    default_args=default_args,
    description='Automated continuous query execution on Snowflake RAW schema',
    schedule_interval='@hourly',
    catchup=False
) as dag:

    # Task: Verify row count in target raw table
    check_orders_count = SnowflakeOperator(
        task_id='check_orders_count',
        sql='SELECT COUNT(*) FROM ECOMMERCE_DB.RAW.ORDERS;',
        snowflake_conn_id='snowflake_conn',
    )

    check_orders_count
```

3. Validate DAG Registration

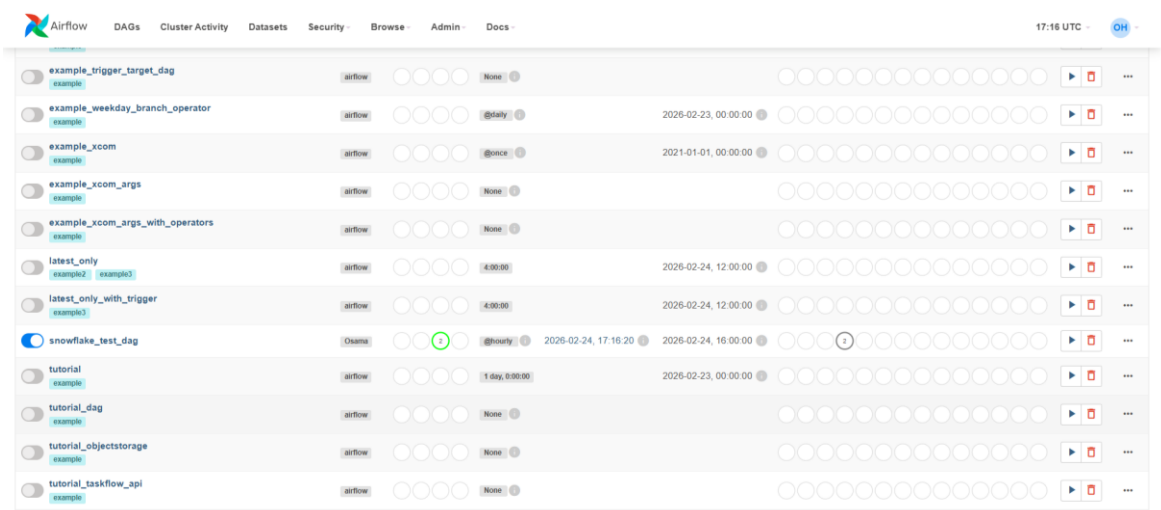
Verify that Apache Airflow correctly parses the new DAG file without syntax or dependency errors:

airflow dags list

(Ensure snowflake test dag appears in the list of recognized DAGs).

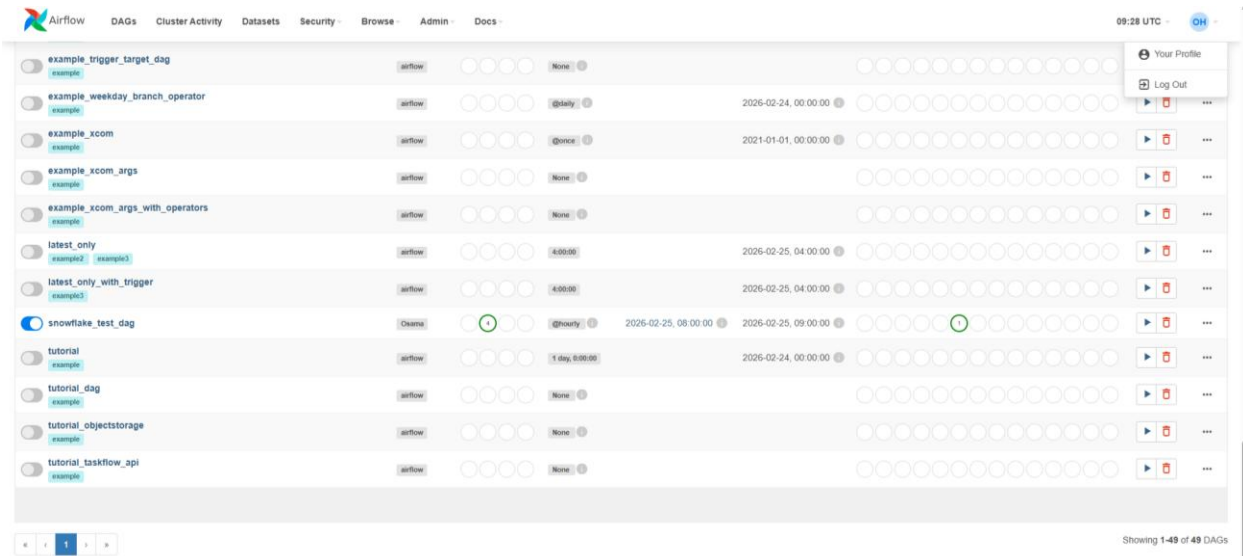
Step 5: DAG Execution & Success Verification

1. Open the Airflow Web UI (<http://<your-public-ip>:8080>) and perform a hard refresh (**Ctrl + F5**).
2. Locate `snowflake_test_dag` and toggle the activation switch to **ON**.
3. Click the **Trigger DAG** (Play) button on the right side to manually trigger execution.



Monitoring Execution

- **Queued State (Yellow):** If tasks remain stuck in queued status, ensure the scheduler service is actively running (airflow scheduler -D).
- **Success State (Dark Green):** Indicates successful connection to Snowflake, statement execution, and response capture.



Phase 3 Summary

- **Orchestrator:** Apache Airflow (v2.8.1) deployed on EC2
- **Connection Layer:** SnowflakeOperator utilizing snowflake_conn
- **Automation Artifact:** snowflake_test_dag (snowflake_automation.py)
- **Access Control:** Port 8080 custom inbound security rule with Admin RBAC
- **Status:** Operational & Validated End-to-End Orchestration

PHASE 4: DATA TRANSFORMATION, DEDUPLICATION & DBT CORE INTEGRATION

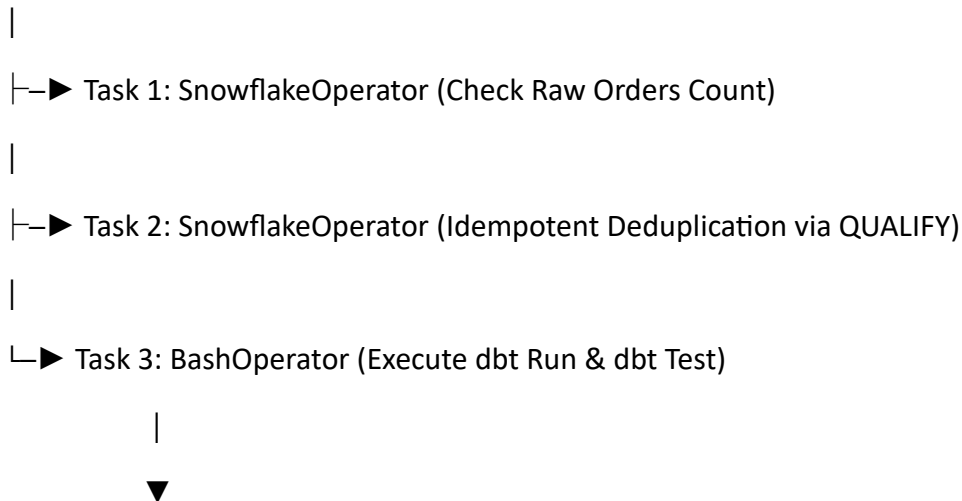
Overview

Phase 4 implements the Data Transformation Layer (Analytics Engineering) of the pipeline. It transitions raw, streamed data (RAW.ORDERS, RAW.CUSTOMERS) into clean, deduplicated, and tested data models stored in an isolated ANALYTICS schema.

This phase evolves the pipeline architecture by introducing **dbt Core (data build tool)** integrated directly into Apache Airflow using BashOperator. The pipeline establishes idempotent ingestion, automated data testing (uniqueness and non-null constraints), and complete software engineering practices (version control, schema isolation, and testing) for data transformations.

Phase 4 Architecture Workflow

[Airflow DAG Orchestrator]



[Snowflake ANALYTICS Schema] —> [Clean Table: STG_ORDERS]

Step 1: Initial In-Warehouse Deduplication Strategy

To guarantee idempotency and eliminate duplicated records caused by continuous streaming drops, the pipeline utilizes window functions (QUALIFY ROW_NUMBER()) and INSERT OVERWRITE semantics.

1. Provision Analytics Table in Snowflake

Execute the following DDL in Snowflake to set up the raw analytics target table:

```
USE DATABASE ECOMMERCE_DB;
USE SCHEMA RAW;

CREATE OR REPLACE TABLE ECOMMERCE_DB.RAW.ANALYTICS_ORDERS (
  ORDER_ID NUMBER,
  CUSTOMER_NAME STRING,
  ITEM_NAME STRING,
  PRICE NUMBER,
  INSERTED_AT TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);
```

2. DML Deduplication Logic

The window function strategy extracts only the first occurrence per ORDER_ID:

```
INSERT OVERWRITE INTO ECOMMERCE_DB.RAW.ANALYTICS_ORDERS (  
    ORDER_ID,  
    CUSTOMER_NAME,  
    ITEM_NAME,  
    PRICE  
)  
SELECT  
    ORDER_ID,  
    CUSTOMER_NAME,  
    ITEM_NAME,  
    PRICE  
FROM ECOMMERCE_DB.RAW.ORDERS  
QUALIFY ROW_NUMBER() OVER (PARTITION BY ORDER_ID ORDER BY ORDER_ID) = 1;
```

Step 2: Snowflake Environment Configuration for dbt

Before deploying dbt, set up an isolated target schema (ANALYTICS) and grant the necessary access rights to the execution role (SYSADMIN).

```
USE ROLE ACCOUNTADMIN;  
  
-- Create dedicated ANALYTICS schema  
CREATE OR REPLACE SCHEMA ECOMMERCE_DB.ANALYTICS;  
  
-- Grant database & schema access to SYSADMIN  
GRANT USAGE ON DATABASE ECOMMERCE_DB TO ROLE SYSADMIN;  
GRANT USAGE ON SCHEMA ECOMMERCE_DB.ANALYTICS TO ROLE SYSADMIN;  
GRANT CREATE TABLE ON SCHEMA ECOMMERCE_DB.ANALYTICS TO ROLE SYSADMIN;  
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA ECOMMERCE_DB.ANALYTICS TO ROLE SYSADMIN;
```

Step 3: dbt Core Setup & Profile Configuration

1. Install dbt-snowflake Adapter

Inside your active Python virtual environment (airflow_env) on the EC2 instance, install the Snowflake dbt adapter:

```
source ~/airflow_env/bin/activate  
pip install dbt-snowflake
```

2. Initialize dbt Project

Initialize the project structure:

```
dbt init ecommerce_dbt
```

3. Configure Connection Profile (~/.dbt/profiles.yml)

Populate the dbt profile with Snowflake database credentials:

```
ecommerce_dbt:
  target: dev
  outputs:
    dev:
      type: snowflake
      account: EJPKLNN-SKB17371
      user: TAHIRHASHMI90
      password: "<your_snowflake_password>"
      role: SYSADMIN
      warehouse: COMPUTE_WH
      database: ECOMMERCE_DB
      schema: ANALYTICS
      threads: 1
      client_session_keep_alive: False
```

4. Test dbt Connection

Verify connection validity between dbt and Snowflake:

```
cd ecommerce_dbt
dbt debug
```

(Ensure Connection test: [OK connection ok] is returned).

Step 4: dbt Data Model & Test Suite Development

1. Define Sources File (models/sources.yml)

Declare the raw database sources and define validation expectations on the landing data:

```
version: 2
sources:
  - name: raw_data
    database: ECOMMERCE_DB
    schema: RAW
    tables:
      - name: orders
        columns:
          - name: order_id
            tests:
              - not_null
              - unique
```

2. Create Staging Model (models/stg_orders.sql)

Define the SQL transformation logic as a software model:

```
{{ config(materialized='table') }}
with raw_orders as (
  select * from {{ source('raw_data', 'orders') }}
)

select
  order_id,
  customer_name,
  item_name,
  price
from raw_orders
qualify row_number() over (partition by order_id order by order_id) = 1
```

3. Define Clean Model Test Suite (models/schema.yml)

Assert quality controls on the transformed staging table:

```
version: 2
models:
  - name: stg_orders
    columns:
      - name: order_id
    tests:
      - unique
      - not_null
```

4. Compile and Run dbt Models & Tests

Execute model compilation and data assertion checks:

```
# Materialize models in Snowflake
dbt run

# Run automated tests on transformed model
dbt test --select stg_orders
```

Execution Test Results

- not_null_stg_orders_order_id: PASS (No null keys detected)
- unique_stg_orders_order_id: PASS (Deduplication engine eliminated 100% of duplicates)

Step 5: End-to-End Airflow Orchestration with dbt

Update the main Apache Airflow DAG (~/airflow/dags/snowflake_automation.py) to orchestrate Snowflake tasks and dbt transformations sequentially using BashOperator.

```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.providers.snowflake.operators.snowflake import SnowflakeOperator
from airflow.operators.bash import BashOperator

default_args = {
    'owner': 'Osama',
    'depends_on_past': False,
    'start_date': datetime(2026, 4, 24),
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

with DAG(
    'snowflake_test_dag',
    default_args=default_args,
    description='End-to-end Snowflake Ingestion, Deduplication, and dbt Transformation DAG',
    schedule_interval='@hourly',
    catchup=False
) as dag:

    # Task 1: Check raw data row counts
    check_orders_count = SnowflakeOperator(
        task_id='check_orders_count',
        snowflake_conn_id='snowflake_conn',
        sql="SELECT COUNT(*) FROM ECOMMERCE_DB.RAW.ORDERS;"
    )

    # Task 2: Perform raw layer deduplication
    transform_data = SnowflakeOperator(
        task_id='transform_data',
        snowflake_conn_id='snowflake_conn',
        sql="""
        CREATE OR REPLACE TABLE ECOMMERCE_DB.RAW.ANALYTICS_ORDERS AS
        SELECT * FROM ECOMMERCE_DB.RAW.ORDERS
        QUALIFY ROW_NUMBER() OVER (PARTITION BY ORDER_ID ORDER BY ORDER_ID) = 1;
        """
    )

    # Task 3: Trigger dbt transformations via BashOperator
    run_dbt_transformations = BashOperator(
        task_id='run_dbt_transformations',
        bash_command='cd /home/ec2-user/ecommerce_dbt && /home/ec2-user/airflow_env/bin/dbt run',
    )

    # Task Execution Sequence
    check_orders_count >> transform_data >> run_dbt_transformations
```

Step 6: Pipeline Verification & Quality Checks

Run analytical checks in Snowflake to confirm data hygiene across RAW and ANALYTICS schemas:

```
-- Compare raw row counts vs analytics deduplicated counts
SELECT
  (SELECT COUNT(*) FROM ECOMMERCE_DB.RAW.ORDERS) AS RAW_ORDERS_COUNT,
  (SELECT COUNT(*) FROM ECOMMERCE_DB.ANALYTICS.STG_ORDERS) AS
ANALYTICS_ORDERS_COUNT,
  (SELECT COUNT(*) FROM ECOMMERCE_DB.RAW.ORDERS) - (SELECT COUNT(*) FROM
ECOMMERCE_DB.ANALYTICS.STG_ORDERS) AS DUPLICATES_REMOVED;

-- Verify 100% uniqueness in clean staging table
SELECT
  COUNT(*) AS TOTAL_ROWS,
  COUNT(DISTINCT ORDER_ID) AS UNIQUE_ORDERS
FROM ECOMMERCE_DB.ANALYTICS.STG_ORDERS;
```

Phase 4 Summary

- **Transformation Tool:** dbt Core (dbt-snowflake) integrated via Airflow BashOperator
- **Data Quality Framework:** Automated schema assertions (unique, not_null)
- **Deduplication Strategy:** Windowing functions (QUALIFY ROW_NUMBER() OVER (PARTITION BY ORDER_ID))
- **Target Schema:** ECOMMERCE_DB.ANALYTICS (STG_ORDERS)
- **Status:** Complete End-to-End Modern Data Stack (EC2 → Kinesis → S3 → Snowpipe → Snowflake RAW → dbt → Airflow → Snowflake ANALYTICS)

PHASE 5: DATA VISUALIZATION & BUSINESS INTELLIGENCE (AMAZON QUICKSIGHT)

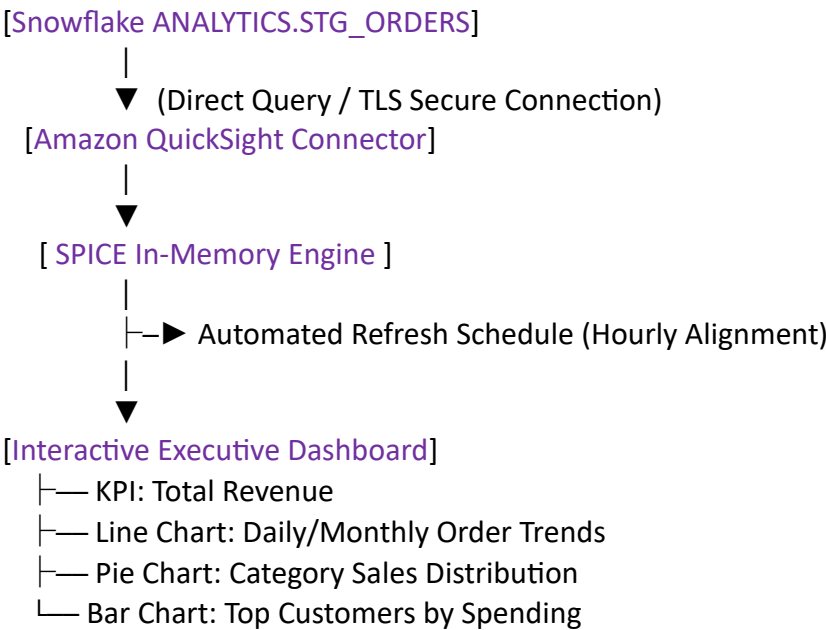
Overview

With clean, transformed, and deduplicated data available in Snowflake (ANALYTICS.STG_ORDERS), Phase 5 completes the end-to-end modern data stack by implementing the Data Visualization and Business Intelligence (BI) layer using **Amazon QuickSight**.

This phase establishes a direct connector between Snowflake and QuickSight, imports data into SPICE (Superfast, Parallel, In-memory Calculation Engine) for high-performance querying,

constructs key e-commerce analytical visuals, and configures automated scheduled refreshes aligned with the upstream Airflow orchestration engine.

Phase 5 Visualization Architecture Workflow



Step 1: QuickSight Setup & Snowflake Connection

Establish an enterprise connection from Amazon QuickSight to the Snowflake Data Warehouse to enable secure data extraction.

1. Navigate to QuickSight Datasets

1. Log into the **AWS Management Console** and launch **Amazon QuickSight**.
2. From the left navigation menu, navigate to **Datasets** and click **New Dataset**.

2. Configure Snowflake Data Source

Select **Snowflake** from the list of external data sources and populate the connection parameters:

Field	Configuration Parameter / Value
Data Source Name	Snowflake_Ecommerce_DS
Database Server	<your_account_identifier>.snowflakecomputing.com
Warehouse	COMPUTE_WH
Database	ECOMMERCE_DB
Schema	ANALYTICS
Authentication	Username / Password (or Dedicated Service Role)

3. Click **Validate Connection** to ensure network connectivity, then click **Create Data Source**.

Step 2: Dataset Preparation & SPICE Ingestion

Import the target table into QuickSight's in-memory storage layer for ultra-fast dashboard responsiveness.

1. **Select Target Model:** Select the transformed dbt model STG_ORDERS (or ANALYTICS_ORDERS) from the schema table list.
2. **Edit & Preview Data:**
 - Click **Edit/Preview Data** to review column metadata.
 - Verify schema data types (e.g., ORDER_ID as Integer/String, PRICE as Decimal/Currency, INSERTED_AT as Date/Timestamp).
3. **Select Import Mode:** Choose **Import to SPICE** (In-Memory) to decouple BI analytical load from Snowflake compute resources and reduce query latencies.
4. Click **Save & Visualize**.

Step 3: Analytical Visuals & Executive Dashboard Design

Construct an interactive executive dashboard covering critical E-Commerce Key Performance Indicators (KPIs).

1. Total Revenue (KPI Card)

- **Visual Type:** KPI Card
- **Field Mapping:**
 - **Value:** SUM(PRICE)
- **Business Value:** Provides an instant snapshot of gross platform revenue.

2. Order Volume Trends (Line Chart)

- **Visual Type:** Line Chart
- **Field Mapping:**
 - **X-Axis:** INSERTED_AT (Aggregated by Day or Month)
 - **Value:** COUNT(ORDER_ID)
- **Business Value:** Tracks business growth, seasonal spikes, and order velocity over time.

3. Product Sales Distribution (Pie / Donut Chart)

- **Visual Type:** Pie Chart or Donut Chart
- **Field Mapping:**

- **Group/Color:** ITEM_NAME
 - **Value:** SUM(PRICE) or COUNT(ORDER_ID)
 - **Business Value:** Identifies top-selling product categories and revenue composition.
- #### 4. Top Customers by Spending (Bar Chart / Data Table)
- **Visual Type:** Horizontal Bar Chart or Ranked Table
 - **Field Mapping:**
 - **Y-Axis / Row:** CUSTOMER_NAME
 - **X-Axis / Value:** SUM(PRICE)
 - **Sort:** Descending by PRICE (Top 10 Filter Applied)
 - **Business Value:** Highlights high-value customers (VIPs) for targeted marketing and retention strategies.



Step 4: Automated Refresh Schedule Configuration

To maintain sync with the upstream Airflow orchestration pipeline (which ingests and transforms data hourly), set up an automated SPICE data refresh.

1. Navigate to **Datasets** in the QuickSight Console and select Snowflake_Ecommerce_DS.
2. Select the **Refresh** tab and click **Add schedule**.

3. Configure the schedule:
 - **Schedule Type:** Incremental or Full Refresh
 - **Frequency:** Hourly
 - **Timezone:** Coordinated Universal Time (UTC) or Local Time
4. Save the schedule. QuickSight will now automatically pull fresh, transformed data from Snowflake every hour.

Phase 5 Summary & Pipeline Conclusion

- **BI Platform:** Amazon QuickSight
- **Storage Engine:** SPICE (In-Memory)
- **Core Metrics Tracked:** Gross Revenue, Order Volume Growth, Product Demographics, High-Value Customers
- **Sync Strategy:** Scheduled Hourly Ingestion matching Apache Airflow orchestration cadence
- **End-to-End Stack Completed:**
 - Real-time Ingestion (EC2/Kinesis) → S3 Raw Storage → Snowpipe Ingestion
→ Snowflake (RAW)
 - dbt Transformations & Airflow Orchestration → Snowflake (ANALYTICS)
→ Amazon QuickSight BI